Object Detection with YOLOv5

on the Pascal VOC 2012 Dataset

Introduction to Neural Networks
AIMS Rwanda, Kigali

★ Group 3

- Olusola Timothy Ogundepo
- Lillian Mutesi
- Lucas M. Razafimanantsoa
- Alliance Irigenera

Outline

1. Task Description
2. Dataset
3. Algorithm Description — YOLOv5
4. Model Training
5. Results & Evaluation
6. Advanced Study: Data Augmentation
7. Advantages & Limitations

Task Description

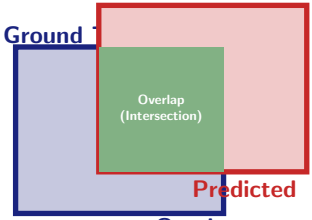
What is Object Detection?

- Identify & localise objects in an image
- Output: **class label** + **bounding box**
- Combines classification and regression
- Evaluated with IoU

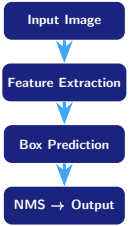
Our Goal

Train **YOLOv5** on Pascal VOC 2012 (20 classes) to accurately detect & classify objects in images.

$$\text{Confidence} = P(\textbf{object}) \times \text{IoU}$$



$$\text{IoU} = \frac{\textbf{Overlap}}{\textbf{Union}}$$



Dataset — Pascal VOC 2012

Pascal VOC 2012

- Benchmark for detection & segmentation
- Real-world photographs, 20 object classes
- YOLO-format annotations:
  `class xc yc w h` (normalised to [0,1])

Class Imbalance

Class imbalance: Person class appears significantly more frequently than others

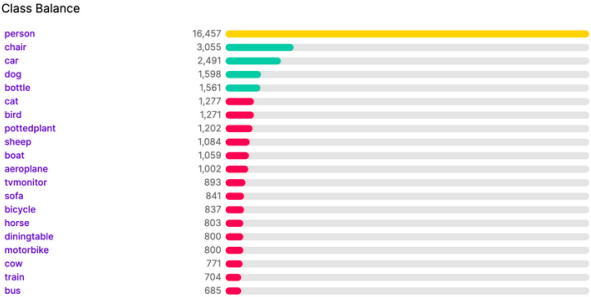


Figure 1: Class distribution in Pascal VOC 2012 training subset

Algorithm — You Only Look Once (YOLO)

Core Idea

YOLO reformulates detection as a **single regression problem**: one forward pass predicts all bounding boxes & class probabilities simultaneously.

Grid

Image divided into $S \times S$ cells. Each cell detects objects whose **centre** falls within it.

Bounding Box

(x_c, y_c, w, h): centre + dimensions, normalised to $[0, 1]$.

Confidence Score

$\text{Conf} = P(\text{obj}) \times \text{IoU}$

Measures presence AND accuracy.

Class Probability

Each cell predicts $P(\text{class}_i \mid \text{object})$ for all C classes.

NMS

Non-Maximum Suppression removes duplicate detections.
Keeps highest-confidence box per object.

Speed

Single forward pass
$\Rightarrow$ real-time detection.
No region proposals needed.

YOLO Algorithm Illustration

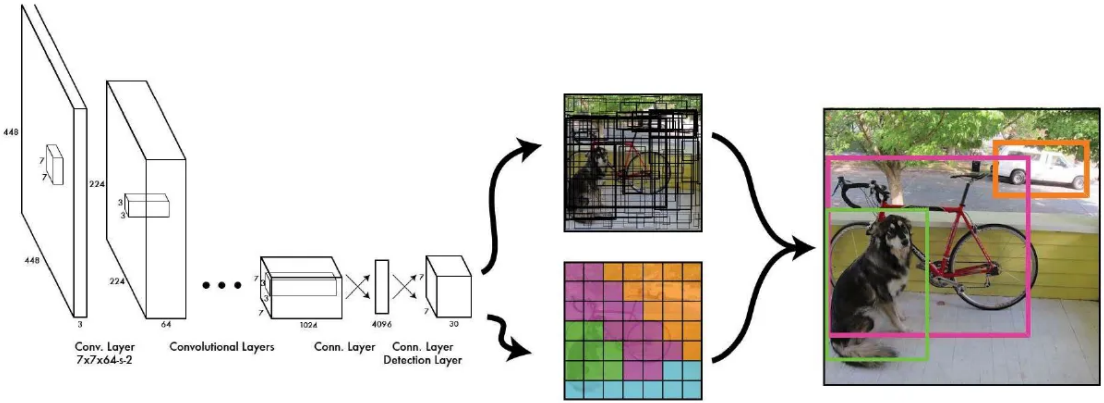
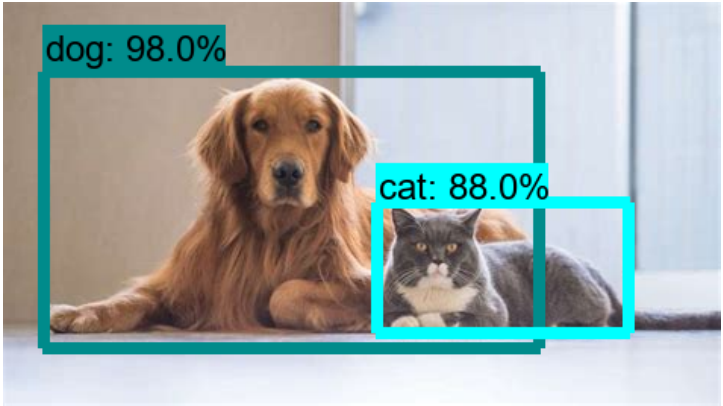


Figure 2: YOLO single-pass detection pipeline: grid division, box prediction, and NMS

Example of Detection Output



Dog

Confidence: 0.98

Cat

Confidence: 0.88

Figure 3: Sample detection output from our trained YOLOv5 model

Model Training

Hyperparameters

Image size	128×128 px
Batch size	8
Epochs	10
Optimiser	Adam
Learning rate	10^{-3}
Subset size	500 images
Device	CPU

Target encoding (`encode_targets`):
Each ground-truth box is placed into the grid cell whose centre falls within it. The target tensor has shape $[S, S, B \times (5+C)]$.

Training loop per batch:

1. Load batch of images + labels
2. Resize to 128×128
3. Encode ground-truth into $8 \times 8 \times 75$ tensor
4. Forward pass through `YOLODetector`
5. Compute `yolo_loss`
6. `loss.backward()` + `optimizer.step()`
7. Record epoch average loss

Model weights saved to `yolo_voc.pt` after training.

Seed fixed at 42 for reproducibility.

Training Loss Curve

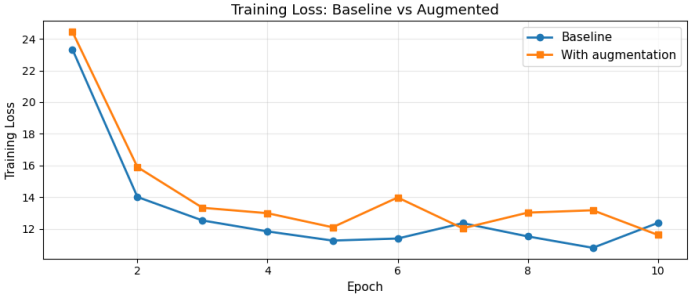


Figure 4: Training Loss: Baseline vs Augmented

Baseline Summary

- Start: **23.34**
- Final: **12.38**
- Reduction: **47.0%**

Observations:

- Steep drop epochs 1–3 (model learns basics)
- Oscillation from epoch 5 (small batch size = 8)
- Aug. model starts higher (harder, varied inputs)
- Both converge to similar final loss

Evaluation Metrics

Loss Function

$$\mathcal{L} = \lambda_{\text{coord}}\mathcal{L}_{xy} + \lambda_{\text{coord}}\mathcal{L}_{wh} + \mathcal{L}_{\text{obj}} + \lambda_{\text{noobj}}\mathcal{L}_{\text{noobj}} + \mathcal{L}_{\text{cls}}$$

- **Coordinate loss**: squared error on predicted box centre (x, y), applied only to cells containing an object.
- **Width/height loss**: squared error on $\sqrt{w}$ and $\sqrt{h}$, giving less sensitivity to large boxes.
- **Object confidence loss**: squared error for cells that contain an object.
- **No-object loss**: squared error on confidence for empty cells, weighted by λ_{noobj}.
- **Class loss**: squared error on class predictions, computed only for cells containing an object.

Results

Set	Mean YOLO Loss
Training subset	≈ 12.4128
Validation (full)	≈ 12.6849
Training subset (augmented)	≈ 11.1727
Validation (augmented)	≈ 13.2454

Advanced Study: Data Augmentation

Motivation

With only 500 training images the model can memorise specific pixel patterns. Augmentation introduces controlled variation at load time.

Transforms applied:

- `RandomHorizontalFlip`$(p = 0.5)$
- `ColorJitter`(brightness$= 0.3$, contrast$= 0.3$, saturation$= 0.2$)
- `RandomRotation`(degrees$= 10$)

Experimental design:
Train a **second model** (`model_aug`) from scratch; same architecture, same hyperparameters, same 500 images, only the transform pipeline differs.

Known Limitation

Horizontal flip mismatch:

The flip is applied to the *pixel tensor* only. The bounding-box coordinates are **not mirrored** $(c_x \not\to 1 - c_x)$.

This introduces a label mismatch for ≈50% of augmented samples.

Despite this, colour jitter and rotation still provide useful diversity; the experiment is a valid *proof-of-concept*.

A correct fix: pass image *and* boxes together through a joint augmentation pipeline.

Advantages & Limitations of YOLOv5

Advantages

- **Real-time**: single forward pass, faster than region-based methods
- **Global reasoning**: processes entire image; captures context, fewer false positives
- **End-to-end**: single regression problem, no multi-stage complexity

Limitations

- **Small subset**: 500 of 13,690 images; weak generalisation
- **Low resolution** (128×128); misses small objects
- **CPU only** (no GPU); forced simpler architecture
- **Only 10 epochs**; model did not fully converge
- **Flip-box mismatch**: corrupts $\sim$50% of aug. labels
- **Class imbalance**: person dominates; biased detections
- **Loss-only eval**: no mAP reported

Possible Improvements

1. Train on **all 13,690** images
2. **GPU** + higher resolution (416×416)
3. **Fix horizontal flip**: mirror $c_x \to 1 - c_x$ with joint image+box transform
4. **Anchor clustering** via k-means on ground-truth boxes
5. **LR scheduler** (e.g. cosine annealing)
6. Report **mAP@0.5** for standard benchmark comparison
7. **Focal loss** to address class imbalance
8. **Transfer learning** from a pretrained backbone

Conclusion

Summary

- Built a **YOLO detector from scratch** in PyTorch (backbone, head, loss function)
- Trained on Pascal VOC 2012 (20 classes) with a 500-image subset
- Loss reduced by **47%** over 10 epochs (23.34 $\rightarrow$ 12.38)
- Conducted a **data augmentation study** with a second model for comparison
- Honestly discussed all limitations and proposed concrete improvements

YOLO's strength is speed and end-to-end simplicity; scaling to full data with GPU training would close the accuracy gap.

Thank you! **Questions welcome.**